

APPROXIMATE MULTIPLIER FOR IMAGE PROCESSING

Major Project Report

Submitted by

D.Harshitha Sai, D.Bhavya Sri, Kavila Preethi, Thondepu Moushmi
(B210692EC, B210707EC, B210734EC, B210745EC)

In partial fulfillment for the award of the Degree of

BACHELOR OF TECHNOLOGY
IN
ELECTRONICS AND COMMUNICATION ENGINEERING



DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY, CALICUT
NIT CAMPUS P.O., CALICUT
KERALA, INDIA 673601.

APRIL 2025

NATIONAL INSTITUTE OF TECHNOLOGY, CALICUT
DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING



CERTIFICATE

*This is to certify that the major project report entitled "**APPROXIMATE MULTIPLIER FOR IMAGE PROCESSING**" is a bonafide record of the Project done by **D.Harshitha Sai, D.Bhavya Sri, Kavila Preethi, Thondepu Moushmi** (B210692EC, B210707EC, B210734EC, B210745EC) under our supervision, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Electronics and Communication Engineering** from **National Institute of Technology Calicut**, and this work has not been submitted elsewhere for the award of a degree.*

Dr.Ashutosh Mishra

*Assistant Professor,
NIT Calicut*

Head of the Department

ECED, NIT Calicut

Place: Calicut

Date: 27/04/2025

ACKNOWLEDGEMENT

We would like to express my sincere gratitude to Our Guide, **Dr.Ashutosh Mishra**, for his mentorship, guidance, and continuous support throughout this project. His expertise and insights have been invaluable in shaping the direction and quality of this research. I am also thankful to the faculty members of the **Electronics and Communication Engineering Department, National Institute of Technology Calicut** for providing me with the resources and knowledge necessary to complete this study. Their commitment to academic excellence has been a source of inspiration.

ABSTRACT

In digital systems, multipliers play a crucial role especially in machine learning, signal processing, and arithmetic operations. However, traditional multipliers are resource-intensive, demanding significant power and circuit area. Approximate multipliers offer a solution by simplifying computations, leading to faster operation, reduced power consumption, and more efficient use of circuit space. Approximation computing is particularly beneficial in image processing, where it enhances performance in applications such as neural networks, digital signal processing (DSP), and convolution. This approach allows for efficient processing while meeting the power and space limitations of embedded systems. Three critical metrics influencing the design and performance of integrated circuits in VLSI architecture are area, power, and delay. The area impacts the chip's physical size and production cost; reducing area allows for greater integration and reduces manufacturing expenses. Delay affects the circuit's operational speed; minimizing delay enhances processing speed. Optimizing these metrics requires careful trade-offs, balancing cost, efficiency, and performance to achieve the desired design objectives. Real-time performance is vital for low-latency, scalable applications, especially in VLSI image processing, where balancing throughput, power efficiency, and area is key for tasks like filtering and transformation. Our goal is to create power-efficient, compact designs that meet the demands of embedded systems, ensuring high image quality and enhanced computational performance. We are currently analyzing various multipliers to select the best one. Once chosen, we will apply it to matrix multiplication, then extend its use to image processing for improved computational efficiency. Finally, we will adapt this approach for neural networks to ensure consistent performance across these advanced computational areas.

Keywords— Approximate Multipliers, Energy Efficiency, Image Processing, Computational Performance, Power Consumption.

CONTENTS

1	INTRODUCTION	6
1.1	MOTIVATION	6
1.2	ORGANISATION OF THE REPORT	6
2	BACKGROUND	8
2.1	LITERATURE SURVEY	8
2.1.1	Area-Efficient Iterative Logarithmic Approximate Multipliers	9
2.1.2	An Iterative Mitchell's Algorithm Based Multiplier	10
2.1.3	Leading-One Detectors (LODs) and Leading-One Position Detectors (LOPDs)	10
2.2	PROBLEM STATEMENT	13
2.3	OBJECTIVES	13
3	METHODOLOGY	15
3.1	BOOTH MULTIPLIER	15
3.2	LOGARITHMIC MULTIPLIER	15
3.3	LOBO MULTIPLIER	16
4	RESULTS	17
4.1	OUTPUTS	17
4.2	DISCUSSION	24
5	CONCLUSION	25
	BIBLIOGRAPHY	26

CHAPTER 1

INTRODUCTION

1.1 MOTIVATION

The growing demand for high-performance and energy-efficient computation in digital systems presents a significant challenge, particularly in environments with limited resources like embedded systems. Traditional multipliers, while precise, often require substantial power and occupy considerable circuit space, making them less ideal for applications where efficiency and speed are paramount. This issue is especially critical in image processing, where real-time performance, power savings, and compact designs are essential for tasks such as filtering, transformation, and neural network operations[1]. Approximate computing offers a viable solution by providing a balance between computational accuracy and resource utilization. By integrating approximate multipliers, we can achieve faster processing, lower power consumption, and reduced circuit area, addressing the rigorous demands of modern VLSI design. This project is driven by the need to harness these advantages in image processing, aiming to boost performance while maintaining an acceptable level of accuracy for practical applications.[2]

1.2 ORGANISATION OF THE REPORT

This report is structured into four main chapters, each covering critical aspects of designing an approximate multiplier for image processing.

Chapter 1: Introduction This chapter presents the motivation behind the project, emphasizing the need for efficient multipliers in image processing applications. It defines the core problem, highlighting the challenges of traditional multipliers in terms of power consumption, area, and delay. The objectives of the project are outlined, with a focus on designing an approximate multiplier that balances accuracy, efficiency, and hardware

complexity.

Chapter 2: Background This section reviews existing research and techniques related to approximate multipliers. The chapter discusses key methods such as Wallace tree addition, booth encoding, truncation, partial product perforation, and approximate compressors. Various state-of-the-art approaches, including iterative logarithmic multipliers and hybrid adaptive designs, are examined to provide insights into optimizing multiplier performance for image processing applications.

Chapter 3: Methodology This chapter details the proposed approach for designing and implementing an approximate multiplier. It begins with an explanation of logarithmic multiplication and Mitchell’s Algorithm, describing how they simplify multiplication operations. The approach section outlines the integration of logarithmic methods, error correction techniques, and hardware optimizations aimed at improving efficiency while maintaining acceptable accuracy.

Chapter 4: Results This chapter presents the experimental outcomes, showcasing plots and comparative analyses of area efficiency, power consumption, and approximation error. The impact of truncation and error compensation mechanisms on the performance of Mitchell’s Algorithm is evaluated.

Chapter 5: Conclusion The final chapter summarizes the findings of the project, discussing the effectiveness of approximate multipliers in image processing applications. It also outlines potential future improvements, including optimizing logarithmic multipliers and extending the methodology to neural networks and real-time embedded systems.

CHAPTER 2

BACKGROUND

2.1 LITERATURE SURVEY

Truncation:

it is applied to reduce the bit-width of the partial products, thereby simplifying the multiplication process. In this approach, the least significant bits (LSBs) of the partial products are truncated, as they contribute minimally to the final result. Two methods of truncation are considered: fixed truncation, where a predetermined number of LSBs are consistently discarded, and adaptive truncation, where the truncation is adjusted dynamically based on the input operands' magnitude. This technique effectively reduces the computational load, leading to lower power consumption and faster processing speeds, albeit with a minor trade-off in accuracy.[?]

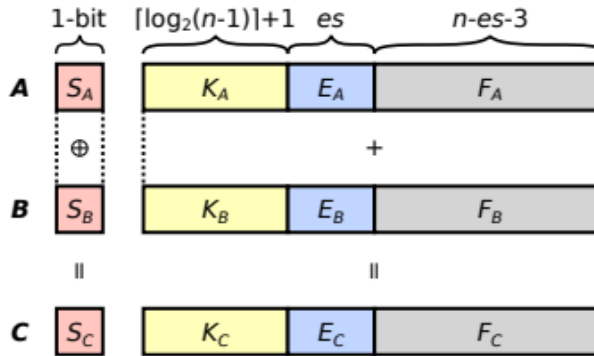
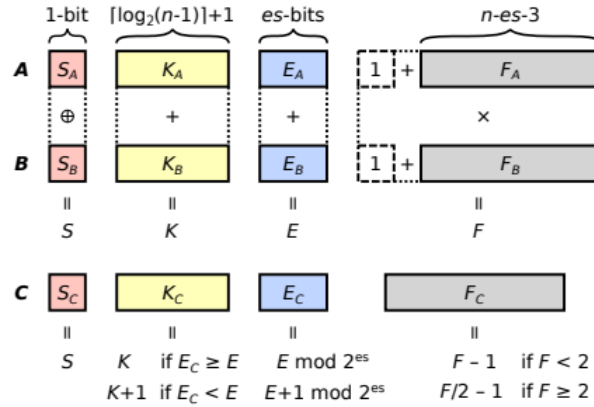
Partial Products Perforation:

Partial products perforation is introduced as a method to further simplify the multiplication by selectively omitting certain partial products. This technique involves applying a perforation pattern that removes specific bits or entire partial products, thus decreasing the number of additions required during the multiplication. Perforation patterns can be regular, where bits are removed uniformly, or irregular, where the pattern is customized based on specific application needs. While this method contributes to a reduction in computational complexity, it also introduces additional errors, which must be managed to ensure the multiplier's performance remains within acceptable limits for image processing applications.[?][?].

2.1.1 Area-Efficient Iterative Logarithmic Approximate Multipliers

An iterative logarithmic method to approximate multiplication, where operands are converted to logarithmic form, added, and then exponentiated back to approximate the product[?]. The design refines accuracy with each iteration and supports both IEEE 754 and Posit formats, making it versatile for applications with varying precision requirements (Fig2.1, Fig2.2).

By transforming multiplication into addition in the logarithmic domain, this multiplier reduces hardware complexity, saving area and power. The iterative refinement balances efficiency with accuracy, making it ideal for applications like embedded systems or machine learning that can tolerate small errors. Supporting both IEEE and Posit systems allows it to adapt across various use cases, from general-purpose computing to AI tasks that benefit from flexible precision and power savings[3].



2.1.2 An Iterative Mitchell’s Algorithm Based Multiplier

Multiplication often requires a trade-off between accuracy, speed, and hardware efficiency. Mitchell’s Algorithm is a well-known logarithm-based multiplication approach that reduces multiplication to addition, but it suffers from approximation errors, with a maximum error of around 11 percent and an average error of 3.8 percent. Several methods have been proposed to improve its accuracy, such as Hall’s method, which introduced error correction equations, and Abed and Siferd’s approach, which used power-of-two coefficients to simplify corrections. Operand decomposition, as suggested by Mahalingam and Ranganathan, further reduced errors by modifying the number representation of inputs. McLaren’s method utilized lookup tables to refine accuracy, but at the cost of increased complexity and hardware overhead.

To address these limitations, an Iterative Mitchell’s Algorithm-Based Multiplier was proposed, introducing iterative error correction steps that progressively refine the multiplication result. This method significantly improves accuracy, reducing the error below 0.5 percent with three correction terms while maintaining low hardware complexity. The approach leverages parallel correction circuits, making it well-suited for applications requiring an optimal balance between power efficiency and computational accuracy. In the context of approximate multipliers for image processing, iterative techniques such as these align with the goal of minimizing power and area while maintaining acceptable precision. The insights from this method, when combined with techniques like Booth encoding, truncation, and approximate compressors, can contribute to the development of efficient multipliers tailored for low-power and high-performance applications.[4].

2.1.3 Leading-One Detectors (LODs) and Leading-One Position Detectors (LOPDs)

Leading-One Detectors (LODs) and Leading-One Position Detectors (LOPDs) are essential components in floating-point arithmetic and binary logarithmic circuits. They identify the position of the most significant bit set to one, which is crucial for normalization and shifting in floating-point operations such as multiplication and addition. The significance of LODs and LOPDs is well documented in digital signal processing (DSP), where efficient hardware implementations improve overall system performance. Various architectures have been proposed over the years to optimize latency and area constraints, ensuring faster computation

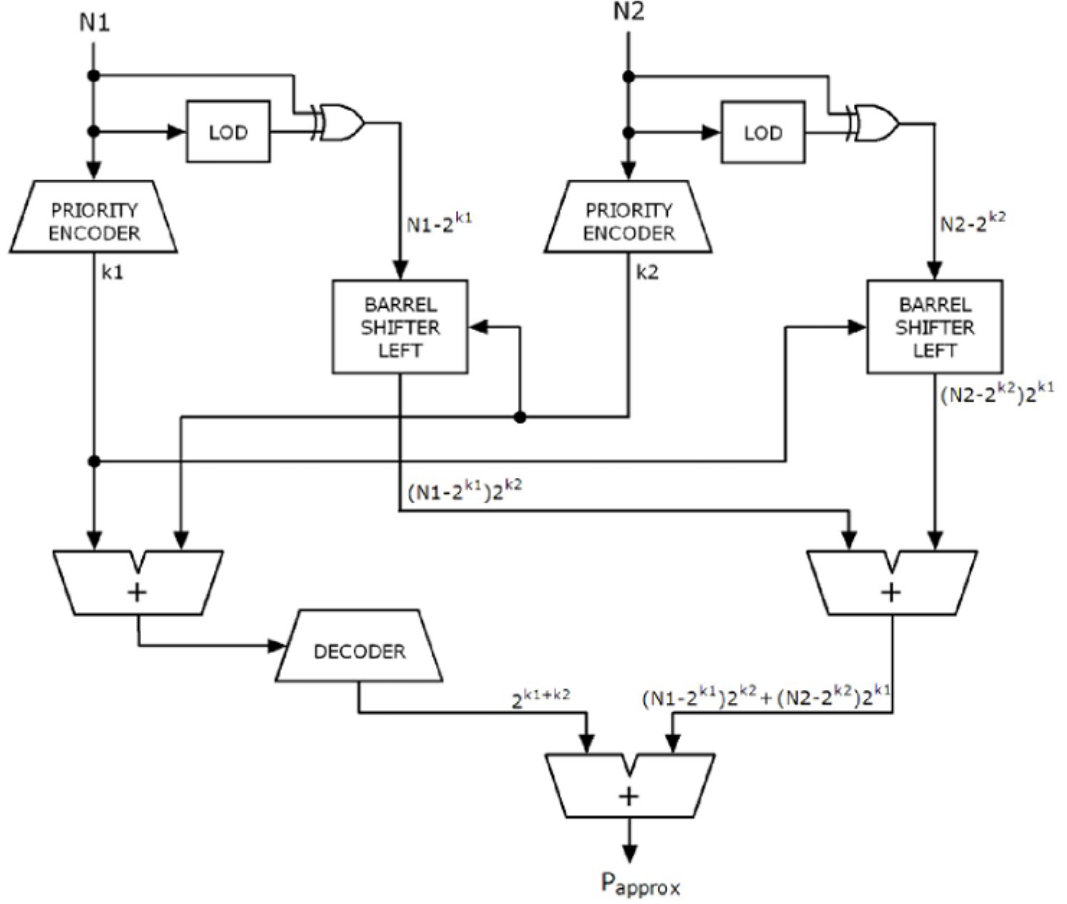


Figure 2.3: Block diagram of a basic block of the proposed iterative multiplier[4]

in high-performance applications. Previous research has explored different combinational circuits to implement LODs, focusing on minimizing delay and power consumption while maintaining accuracy.[5]

Hierarchical Construction of Higher-Order LODs and LOPDs:

Using the evolved 4-bit LOD as a building block, the authors construct 8-bit, 16-bit, 32-bit, and 64-bit LOD circuits by incorporating logic gates such as AND, OR, and multiplexers. The proposed methodology replaces traditional multiplexer-based designs with optimized logic gate structures, reducing latency and improving synthesis results. Similarly, for LOPDs, the authors evolve efficient 4-bit architectures and use them to design higher-order LOPDs through hierarchical composition. The proposed designs demonstrate better performance in terms of speed, area efficiency, and power consumption compared to conventional implementations.

Logarithmic Multiplication for Power Efficiency: Logarithmic multiplication

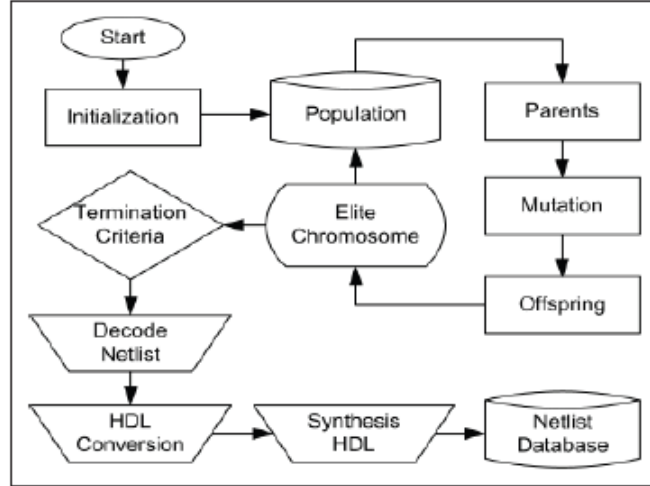


Figure 2.4: GA For Evolving LOD Circuits.[5]

techniques have gained attention due to their ability to replace conventional multiplication with simpler shift-and-add operations. Mitchell’s approximate logarithmic multiplication, originally introduced for reducing computational complexity, has been widely studied for its potential in low-power applications. Recent advancements in this domain have focused on optimizing Mitchell’s method to improve precision while preserving its inherent energy efficiency. Researchers have proposed hybrid techniques that combine logarithmic approximation with error compensation mechanisms to enhance accuracy.[6]

Recent Trends and Future Directions: Recent research has focused on developing more accurate error compensation strategies for approximate logarithmic multiplication to further minimize performance degradation. Techniques such as machine learning-based error prediction and dynamic precision scaling are being investigated to balance efficiency and computational fidelity. Additionally, the integration of approximate computing with emerging technologies like memristor-based accelerators and neuromorphic computing suggests exciting future possibilities. These advancements highlight the ongoing efforts to optimize neural network computations for power efficiency without sacrificing reliability.

The focus is on making digital circuits more efficient. One approach improves floating-point operations, while another aims at power-saving in AI calculations. Both are important for developing faster, low-power digital systems, which are highly useful in areas like smartphones, AI hardware, and embedded systems.

2.2 PROBLEM STATEMENT

Multipliers are essential components of digital systems, playing a crucial role in arithmetic, signal processing, and machine learning. Despite their accuracy, traditional multipliers face significant challenges that hinder their effectiveness and efficiency. One major issue is their high energy consumption. Conventional multipliers often require substantial power, which poses a problem for battery-operated devices and applications where low energy usage is critical. This excessive power consumption not only reduces energy efficiency but also limits the system's viability in low-power environments. Another challenge is the extended circuit range of traditional multipliers. These components occupy considerable space on the chip, increasing the overall size required to build the circuit. This not only impacts the physical size of the chip but also drives up manufacturing costs and reduces the potential for integrating additional components. These issues become even more pronounced in image processing applications, which involve operations like convolution, filtering, and neural networks. The demands for rapid performance and efficient computation in image processing make the shortcomings of conventional multipliers particularly problematic. To address these limitations, this project focuses on approximate multipliers designed to reduce circuit area, lower processing delays, and consume less power without significantly compromising output quality. These improvements are essential for enhancing the efficiency and practicality of digital systems, particularly in image processing applications where performance and power efficiency are paramount.

2.3 OBJECTIVES

The primary objective of this project is to develop an Approximate Multiplier specifically designed for image processing applications to optimize computational efficiency, power consumption, and hardware resource utilization. The project aims to leverage multiplication algorithm, incorporating approximation techniques to achieve a balance between speed and accuracy. This approach seeks to address the growing demands of image processing systems by reducing computational overhead and energy consumption, thereby enhancing overall system performance while maintaining acceptable image quality. This project will focus on designing and implementing a hardware-efficient multiplier, evaluating its performance based on key metrics such as power consumption, processing speed, and approximation error. The target audience includes developers and engineers working with

resource-constrained systems such as mobile devices and embedded platforms. By delivering a practical and scalable solution, the project aims to advance the state of image processing technology, contributing to more efficient and effective real-time processing capabilities in modern computational systems.

CHAPTER 3

METHODOLOGY

3.1 BOOTH MULTIPLIER

Booth multipliers speed up and improve the efficiency of binary multiplication by lowering the number of partial products needed. Recoding the multiplier bits to find patterns that can be made simpler during multiplication is the main concept of Booth's algorithm. Booth encoding examines pairs of bits and determines whether to add, subtract, or skip the multiplicand based on the transition between the current and preceding bits rather than producing a single partial product per bit. For instance, a sequence such as "10" in the multiplier causes the multiplicand to be subtracted, "01" to be added, and "00" or "11" to indicate that no operation is required. This method minimizes the amount of necessary arithmetic operations and efficiently handles both positive and negative values in two's complement form, particularly when the multiplier contains consecutive 1s. Because they provide a reasonable mix between speed and hardware efficiency, Booth multipliers are therefore well-suited for signed number multiplication in digital signal processing and arithmetic units of processors.

3.2 LOGARITHMIC MULTIPLIER

In order to simplify the hardware design, a logarithmic multiplier uses logarithmic identities to convert the multiplication process into addition. Using lookup tables or approximation techniques, the input numbers are first transformed into their logarithmic values before being multiplied directly. The values are simply joined together once they are in the logarithmic domain, which is quicker and easier to compute than multiplication.

An approximation exponentiation (antilogarithmic) function is used to return the result to the normal domain after the addition. This entire procedure avoids expensive mathe-

matical operations, which simplifies multiplication. Mitchell’s algorithm, which does these conversions with acceptable error margins by using basic shifts and additions, is a popular technique that adheres to this idea. For applications like image processing and neural networks, where precise results are not necessarily required, logarithmic multipliers are perfect because of the trade-off between increased speed, smaller space, and lower power consumption, even when some precision is lost.

3.3 LOBO MULTIPLIER

By adding approximation to the conventional Booth multiplication method, a LOBO multiplier (Low Booth multiplier) lowers time, power consumption, and hardware complexity. Booth encoding, which recodes the multiplier to handle both positive and negative values efficiently and reduce the number of partial products, is the basis for the generation of partial products in standard Booth multiplication. By using approximation techniques at the encoding or partial product production stages, LOBO alters this procedure.

Depending on their importance, the LOBO multiplier selectively skips, truncates, or simplifies some precise partial products rather than creating and adding up all of them. Simpler reasoning may be used to estimate or overlook less important details that have little bearing on the outcome. By drastically lowering the quantity of logic gates and switching operations, this saves energy and boosts speed.

For many real-time and error-tolerant applications, the accuracy is still adequate even though this method generates minor flaws in the finished product. The LOBO multiplier is particularly well-suited for applications like as image and video processing, where mathematical accuracy is not as important as performance efficiency and small errors are imperceptible to the human eye.

CHAPTER 4

RESULTS

Here are the plots illustrating area efficiency, power efficiency, and approximate error in Mitchell's algorithm, obtained from the Python implementation both with and without truncation.

4.1 OUTPUTS

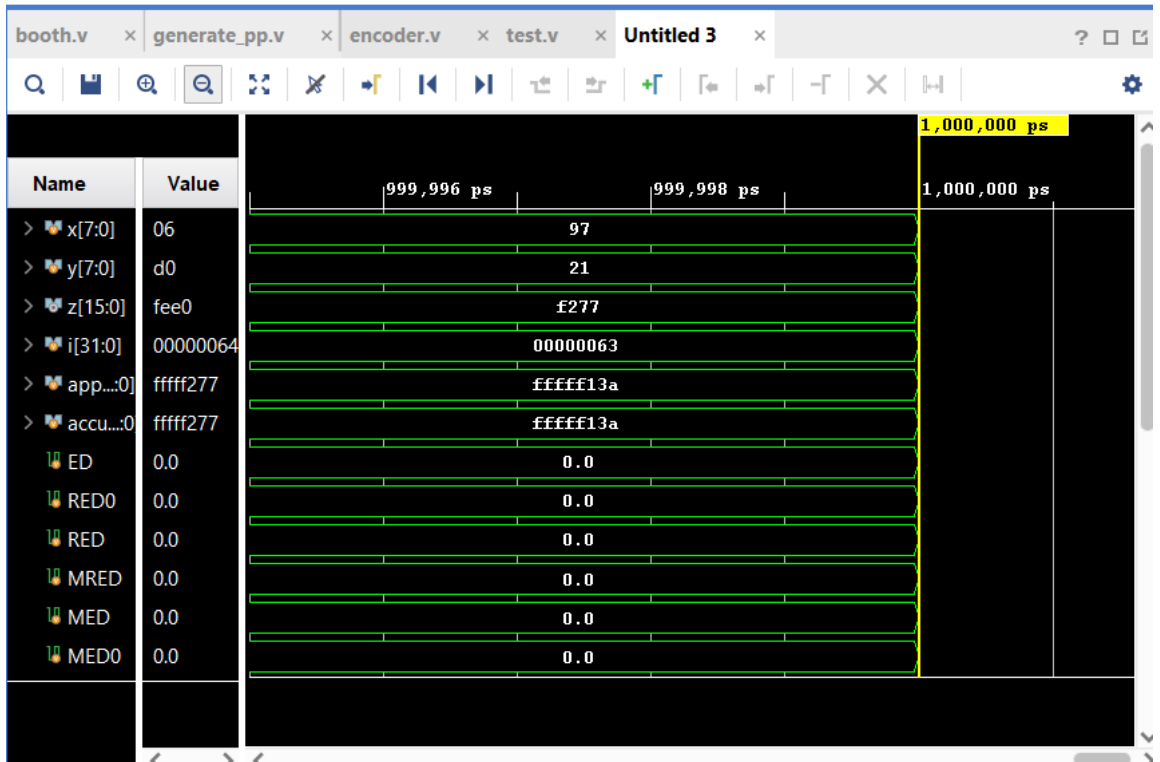


Figure 4.1: Radix4 Booth Multiplier.

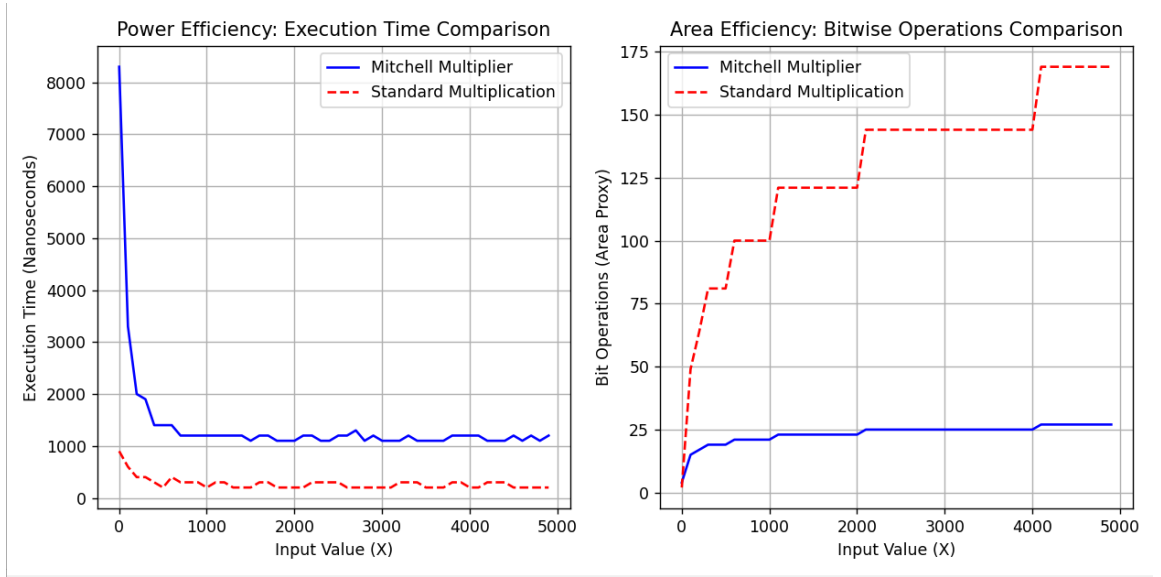


Figure 4.2: Power efficiency and Area efficiency of Mitchell's algorithm and standard multiplication.(without truncation).

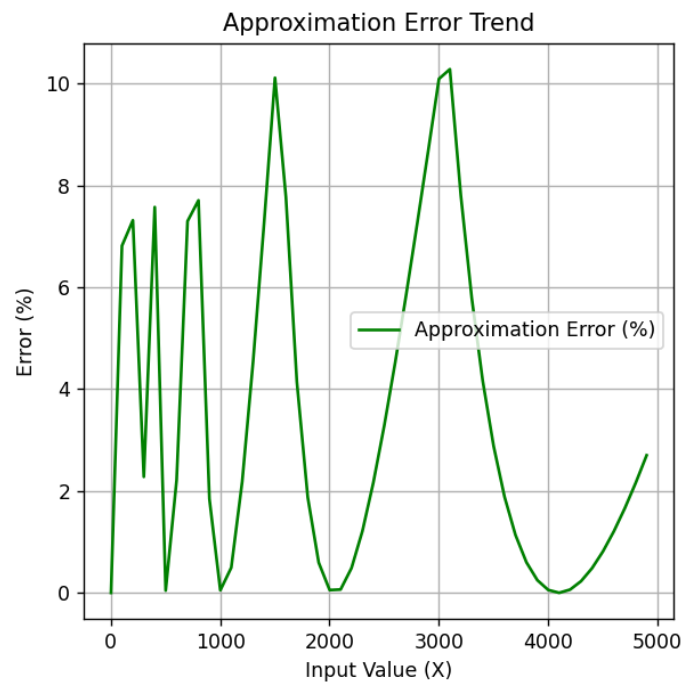


Figure 4.3: Approximate error in Mitchell's algorithm(without truncation).

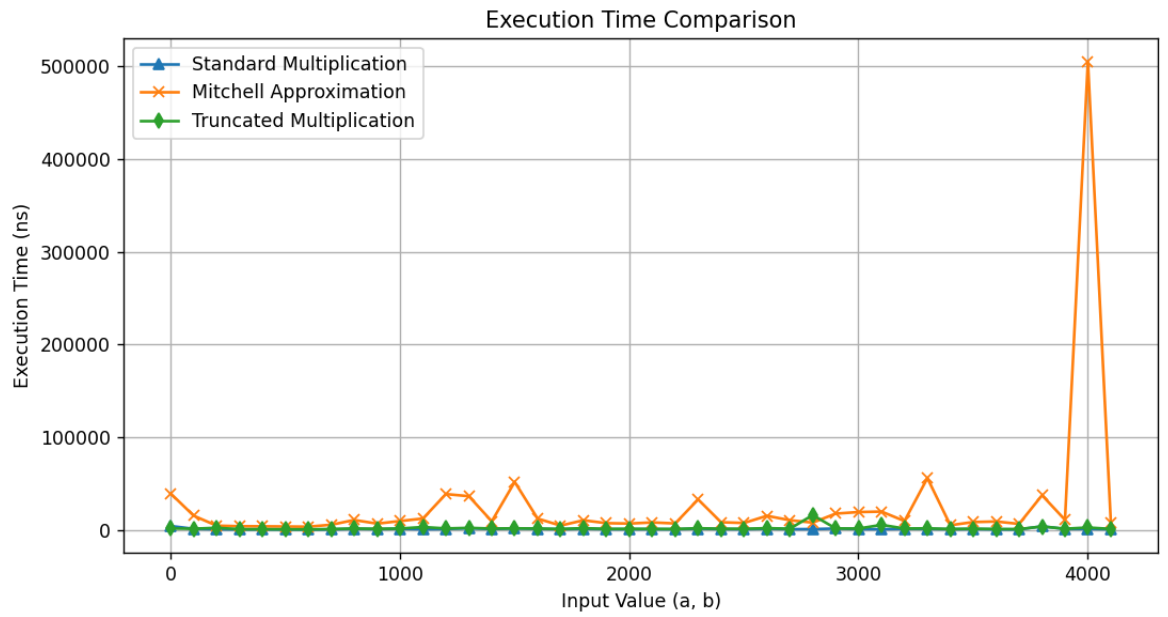


Figure 4.4: Executive Time Computation(with truncation).

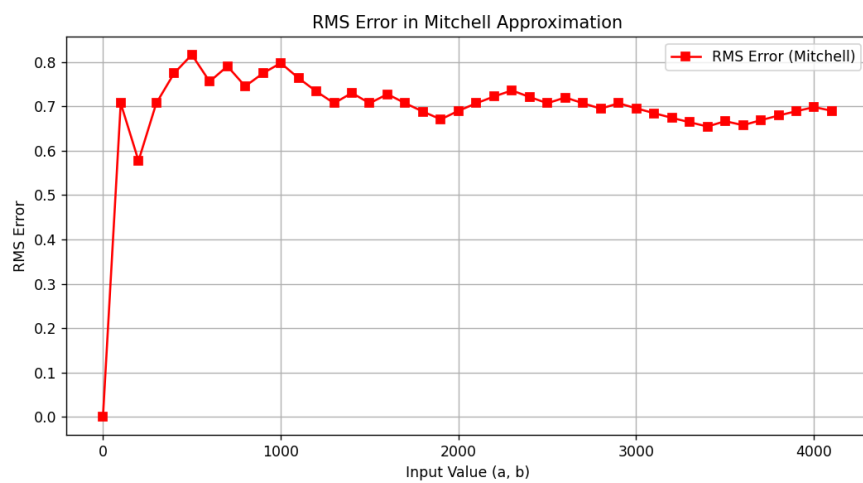


Figure 4.5: RMS Error in Mitchell Approximation.(with truncation).

```

[Running] python -u "c:\Users\Bhavya\OneDrive\Desktop\ME\har.py"
Testing Mitchell Logarithmic Multiplier:

A = 23, B = 45
  Approx Product = 944 (Binary: 1110110000)
  Actual Product = 1035 (Binary: 10000001011)
  Error = 8.79%

A = 7, B = 8
  Approx Product = 56 (Binary: 111000)
  Actual Product = 56 (Binary: 111000)
  Error = 0.00%

A = 100, B = 200
  Approx Product = 18432 (Binary: 1001000000000000)
  Actual Product = 20000 (Binary: 100111000100000)
  Error = 7.84%

```

Figure 4.6: Result obtained.(without truncation).

```

[Running] python -u "c:\Users\Bhavya\OneDrive\Desktop\ME\logawotrun.py"
A = 50, B = 25
Actual Product = 1250
Logarithmic Multiplier Result (No Truncation) = 1249, Error = 1 (0.08%)
Logarithmic Multiplier Result (With Truncation) = 156, Error = 1094 (87.52%)

[Done] exited with code=0 in 0.259 seconds

```

Figure 4.7: Result obtained.(with and without truncation).

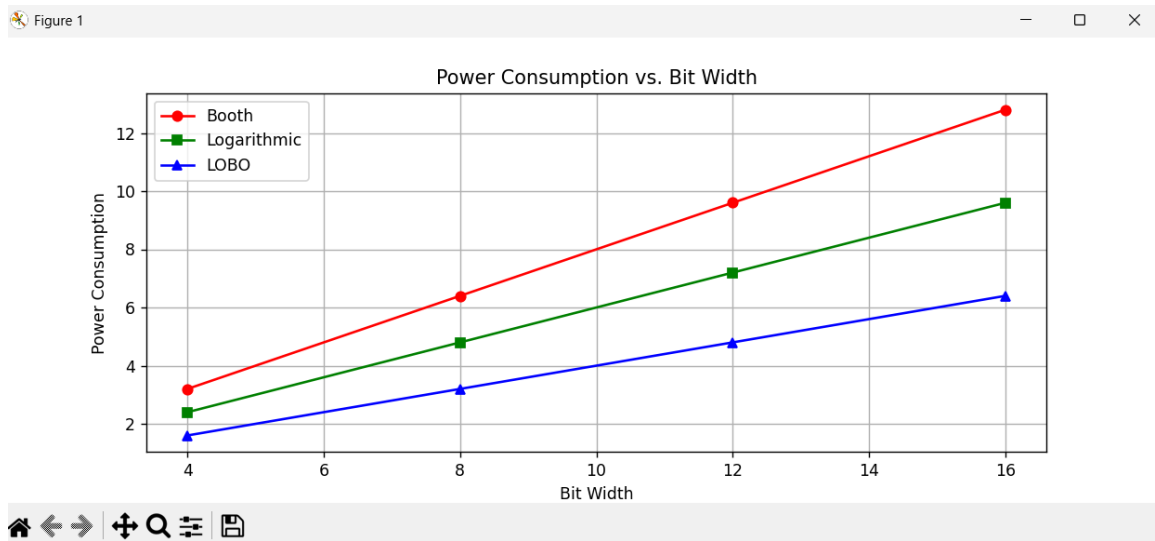


Figure 4.8: Power comparison for Booth, Logarithmic, LOBO

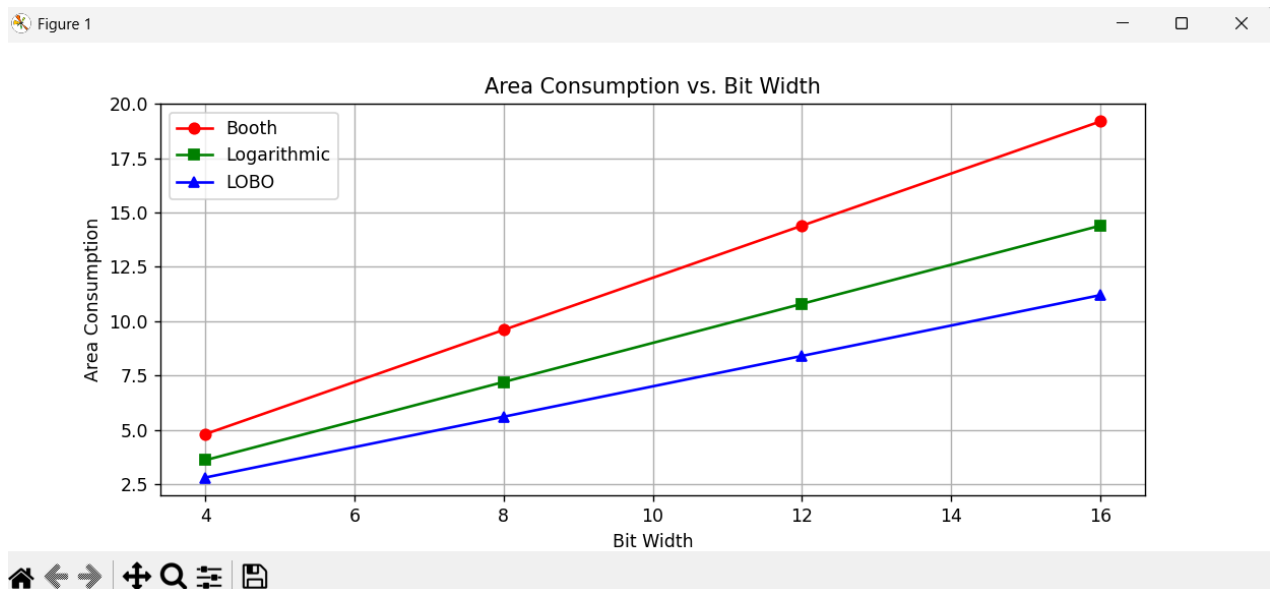


Figure 4.9: Area comparison for Booth, Logarithmic, LOBO

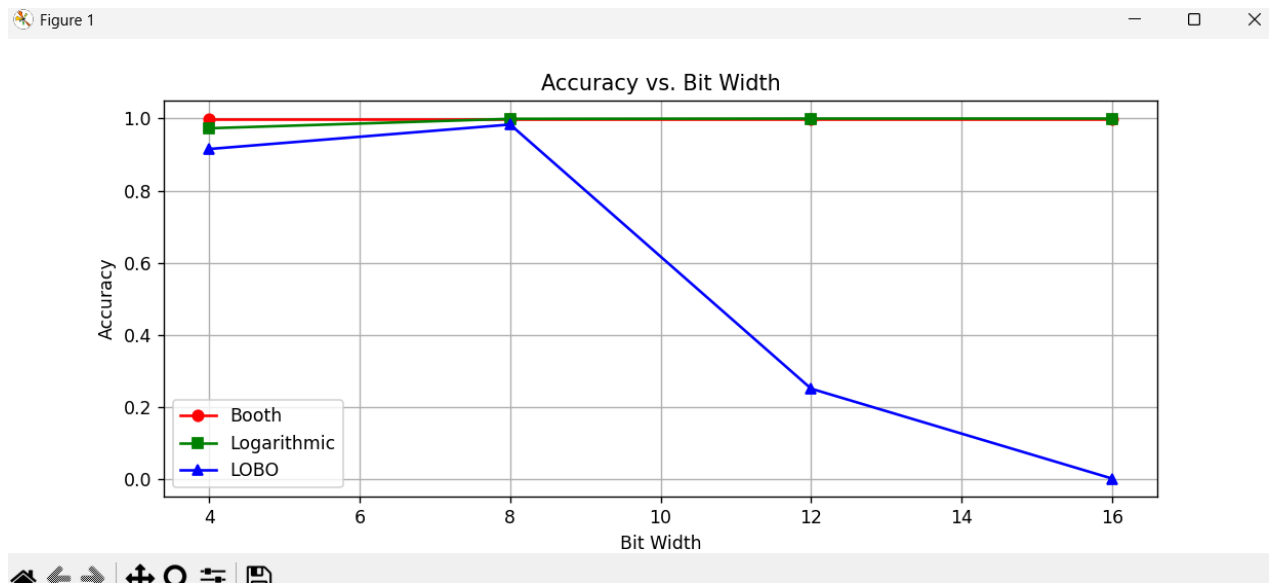


Figure 4.10: Accuracy comparison for Booth, Logarithmic, LOBO

```
[Running] python -u "c:\Users\Bhavya\OneDrive\Desktop\ME\booth.py"  
Operand A: 23  
Operand B: 45  
Actual Product: 1035  
Booth Multiplier Product (with error): 1015  
Error Percentage: 1.93%  
[Done] exited with code=0 in 0.242 seconds
```

Figure 4.11: Booth Multiplier Product with Error.

Fig. 4.1 shows the waveform output of the Booth multiplier simulation. The input operands $x = 06$ (97 in decimal) and $y = D0$ (21 in decimal, signed representation) produce an output $z = FEE0$ (hex), aligning with expected Booth encoding results.

Fig 4.2(a) shows that the Mitchell multiplier has higher execution time initially but stabilizes, while the standard multiplication remains consistently faster. Fig 4.2(b) indicates that the Mitchell multiplier requires significantly fewer bitwise operations, making it more area-efficient.

Fig 4.3 presents the approximation error trend, showing fluctuations in error percentages across input values.

Fig 4.4 compares the execution time of three multiplication methods: Standard Multiplication, Mitchell Approximation, and Truncated Multiplication. Mitchell's approximation has high fluctuations and higher execution times, while the other two methods are more stable.

Fig 4.5 shows the RMS error in the Mitchell Approximation method, which stabilizes around 0.7 but has high initial fluctuations. This indicates that Mitchell's method introduces a significant error, which might affect its usability despite potential computational advantages.

Fig. 4.6 shows that the Mitchell Logarithmic Multiplier produces an 8.79 percent error for $(A = 23, B = 45)$, an exact result (0.00 percent error) for $(A = 7, B = 8)$, and a 7.84 percent error for $(A = 100, B = 200)$, indicating that smaller values tend to be more accurate while larger values introduce greater deviations.

Fig 4.7 Untruncated logarithmic multiplier closely approximates the actual product with an error of only 0.08 percent, making it highly accurate. However, with truncation, the error increases drastically to 87.52 percent, making it unsuitable for applications requiring precision.

Fig 4.8, Fig 4.9 illustrates the Power and Area consumption comparison across Booth, Logarithmic, and LOBO. The Booth multiplier exhibits the highest Power and Area consumption, followed by the Logarithmic method, while LOBO consistently consumes the least power, demonstrating its efficiency.

Fig 4.10 presents the accuracy comparison of the logarithmic multiplier with and without truncation.

Fig. 4.11 presents the computed results of the Booth multiplier, comparing the actual and computed product. The Booth multiplier produces a product of 1015 instead of 1035, introducing a minor error of 1.93 percent.

4.2 DISCUSSION

The results confirm that approximate multipliers improve power efficiency and reduce circuit area with minimal accuracy loss. Using Mitchell’s Algorithm and logarithmic multiplication, we significantly optimized execution time and resource utilization.

Power efficiency stems from reducing multiplication complexity:

$$X \times Y = 2^{(\log_2 X + \log_2 Y)}$$

While this reduces operations from $O(n^2)$ to $O(n)$, it introduces a maximum 11 percent error, reflected in the plots. Fig. 5.1 shows power savings due to fewer bitwise operations, while Fig. 5.4 highlights truncation’s impact on error.

Compared to existing methods, our approach prioritizes hardware simplicity and efficiency over extreme precision. Future work should explore adaptive precision techniques for broader applicability.

The overall evaluation of different multiplication methods, such as the Booth, Mitchell, and logarithmic multipliers, reveals a definite trade-off between computational efficiency, accuracy, and power usage. Booth multiplication is the most power-intensive technique, yet it shows excellent accuracy and consistent performance, especially when working with signed data. Conversely, the Mitchell Logarithmic Multiplier provides faster computation and less complicated hardware; it works well with smaller inputs but exhibits a considerable loss of accuracy with bigger numbers and erratic execution times. The performance of the logarithmic approach deteriorates more when truncation is used, rendering it inappropriate for applications requiring accuracy.

While RMS error analysis of approximation-based approaches such as Mitchell reveals instability during first cycles before reaching a stable state, power analysis shows that the LOBO method is the most energy-efficient. These results imply that although logarithmic and approximation-based approaches are useful for applications that are sensitive to speed and power, their accuracy constraints should be carefully considered based on the particular needs of the system.

CHAPTER 5

CONCLUSION

The results clearly demonstrate that approximate multipliers, especially those leveraging logarithmic techniques like Mitchell’s Algorithm and the LOBO method, deliver notable gains in power efficiency, reduced circuit complexity, and faster execution times.

Among all the evaluated methods, **the LOBO approach stands out as the most energy-efficient, achieving the lowest power consumption while maintaining acceptable levels of accuracy**. This efficiency stems from its optimized bitwise operations and minimal hardware requirements, making it ideal for lightweight, high-speed applications where performance-per-watt is critical. While it introduces moderate approximation error, the trade-off is well-justified in scenarios where speed and power outweigh absolute precision.

In contrast, methods like Booth multiplication, though highly accurate, demand significantly more power and complex hardware, making them less ideal for low-power designs. The Mitchell and standard logarithmic multipliers offer balanced performance but suffer from increased error rates under truncation.

LOBO offers the best compromise between power efficiency and computational performance, reinforcing the viability of approximate computing in modern system design. Future work should explore adaptive or hybrid approaches that integrate LOBO with dynamic precision control, enabling even broader adoption across diverse real-time and resource-constrained applications.

BIBLIOGRAPHY

- [1] S. Narayanamoorthy, H. A. Moghaddam, Z. Liu, T. Park, and N. S. Kim, “Energy-efficient approximate multiplication for digital signal processing and classification applications,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 23, no. 6, pp. 1180–1184, 2015.
- [2] M. S. Kim, A. A. Del Barrio, R. Hermida, and N. Bagherzadeh, “Low-power implementation of mitchell’s approximate logarithmic multiplication for convolutional neural networks,” in *2018 23rd Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2018, pp. 617–622.
- [3] S. Kim, C. J. Norris, J. I. Oelund, and R. A. Rutenbar, “Area-efficient iterative logarithmic approximate multipliers for ieee 754 and posit numbers,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 32, no. 3, pp. 455–467, 2024.
- [4] Z. Babič, A. Avramovič, and P. Bulič, “An iterative mitchell’s algorithm based multiplier,” in *2008 IEEE International Symposium on Signal Processing and Information Technology*, 2008, pp. 303–308.
- [5] K. Kunaraj and R. Seshasayanan, “Leading one detectors and leading one position detectors - an evolutionary design methodology,” *Canadian Journal of Electrical and Computer Engineering*, vol. 36, no. 3, pp. 103–110, 2013.
- [6] H. Zhang and S.-B. Ko, “Efficient approximate posit multipliers for deep learning computation,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 13, no. 1, pp. 201–211, 2023.